

Predicados Numéricos

João Eduardo Montandon
DCC024

Como calcular o tamanho de uma lista em Prolog?

```
mylength([], 0).  
mylength([_|T], Len) :mylength(T, Tlen), Len = Tlen + 1.
```

```
?- mylength([1,2,3], X).
```

Qual o resultado dessa consulta? 😬

```
X = 0+1+1+1.
```

OOPS! 😬

Lembre-se, em Prolog:

- Tudo são termos, constantes ou variáveis.
- A computação se dá pelo casamento de predicados e regras (Unificação).

Então, como transformar `X = 0+1+1+1` em `X = 3` ? 😬

Avaliação Numérica

- Predicados compostos por números podem ter seus termos avaliados.
- Para isso utilizamos o predicado `is` .

```
?- X = 0+1+1+1.  
X = 0+1+1+1.  
? - X is 0+1+1+1.  
X = 3.
```

```
?- Y=X+2, X=1.  
Y = X + 2.  
X = 1.
```

```
?- Y is X+2, X=1.  
🌟🌟🌟
```

Qual será o resultado dessa execução? 😬

Predicado `is`

- `is` é, na verdade, um predicado.
- `X is 1 + 1. ≡ is(X, 1 + 1).`
- Primeiro termo aceita variáveis.
- Segundo termo deve estar instanciado.**

Predicados Numéricos

São predicados aceitos no processo de avaliação.

`+`, `-`, `*`, `/`, `abs(X)`, `sqrt(X)`.

Alguns exemplos.

```
?- X is 2/1.  
X = 2.  
  
?- X is 2.0/1.0.  
X = 2.0.  
  
?- X is 1.0/2.0.  
X = 0.5.  
  
?- X is 1/2. %😬 Interessante ...  
X = 0.5.
```

Comparadores Numéricos

`>`, `<`, `≥`, `=<`, `==`, `=\=`.

Os termos são **avaliados automaticamente**.

```
?- 1 < 2.  
true.  
?- 1+1 == 2.  
true.  
?- 3*2/2 =\= 3.  
false.
```

Igualdade

Qual a diferença entre elas?

```
?- 3 = 2+1.  
😬  
?- 3 is 2+1.  
😬  
?- 3 == 2+1.  
😬
```

- `X=Y` ➡ Unifica `X` e `Y`. Não realiza avaliações.
- `X is Y` ➡ Avalia `Y` e unifica com `X`.
- `X == Y` ➡ Avalia tanto `X` quanto `Y`, retorna `true` se forem numericamente iguais.

Alguns Exemplos

- 1. soma
- 2. mdc
- 3. fact

soma

```
soma([], 0).
soma([H|T], ST) :- soma(T, STT), ST is ST + STT.
```

Como torná-la recursiva de cauda? 😬

mdc

```
mdc(X, Y, D) :- X == Y, D is X.
mdc(X, Y, D) :- X > Y, NewX is X - Y, mdc(NewX, Y, D).
mdc(X, Y, D) :- X < Y, mdc(Y, X, D).
```

fact

```
fact(0, 1).
fact(1, 1).
fact(N, F) :- N > 1, NewN is N-1, fact(NewN, NF), F is N * NF.
```

Problemas De Espaço De Busca

Prolog é uma linguagem que se destaca em problemas cuja solução passa por um **espaço de busca**.

Dê um conjunto de restrições que definam a solução, Prolog executará as combinações necessárias para encontrá-la.

O Problema Das Oito Rainhas

Como organizar oito rainhas em um tabuleiro de xadrez sem que nenhuma fique em xeque?^[1]

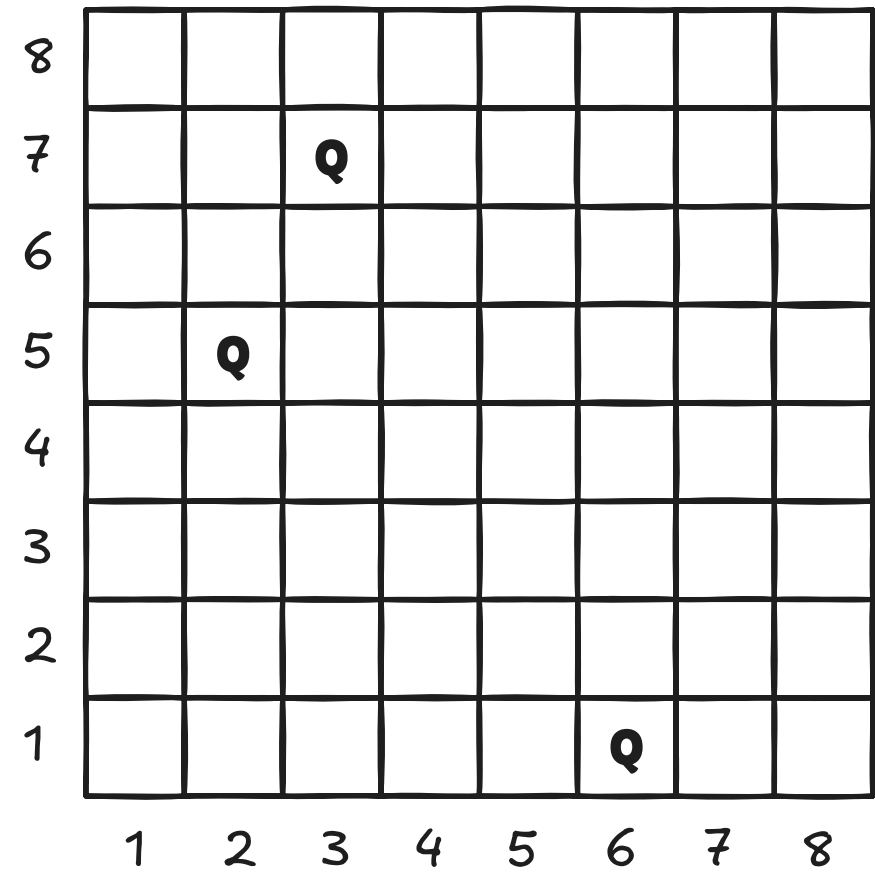
Noções Básicas

- Tabuleiro possui 8x8 de tamanho.
- Uma rainha pode mover verticalmente, horizontalmente, ou em diagonal.
- Duas rainhas estão *em cheque* se compartilharem mesma linha, coluna ou diagonal.

Representação

- A posição de uma rainha é definida por duas coordenadas, **linha** e **coluna**.
- Opção 01: queen(2,5).
- Opção 02: 2/5, lembre-se, Prolog não irá avaliar a não ser que is seja usado.

queens = [2/5, 3/7, 6/1]



Predicado `nocheck`

```
/*
  O predicado recebe uma rainha X/Y e uma lista de rainhas.
  O predicado será resolvido se não houver nenhuma rainha
  na lista que deixe a rainha X/Y em xeque.
*/
nocheck(_, []).
nocheck(X/Y, [X1/Y1 | T]) :- X \= X1,
  Y \= Y1,
  abs(X-X1) \= abs(Y-Y1),
  nocheck(X/Y, T).
```

Predicado `legal`

```
/*
  O predicado gera uma configuração válida de rainhas.
  Uma rainha é válida quando suas coordenadas estiverem
  entre 1 e 8, e não estar em xeque com outras rainhas.
*/
legal([]).
legal([X/Y | T]) :- legal(T),
  member(X, [1,2,3,4,5,6,7,8]),
  member(Y, [1,2,3,4,5,6,7,8]),
  nocheck(X/Y, T).
```

Setup Inicial

```
eightqueens(X) :- length(X, 8), legal(X).
```

```
?- eightqueens(X).
X = [8/4, 7/2, 6/7, 5/3, 4/6, 3/8, 2/5, 1/1] .
```

That's all folks! 🎯

... Será? 🤔

```
?- eightqueens(X).
X = [8/4, 7/2, 6/7, 5/3, 4/6, 3/8, 2/5, 1/1] ;
X = [7/2, 8/4, 6/7, 5/3, 4/6, 3/8, 2/5, 1/1] ;
X = [8/4, 6/7, 7/2, 5/3, 4/6, 3/8, 2/5, 1/1] ;
X = [6/7, 8/4, 7/2, 5/3, 4/6, 3/8, 2/5, 1/1] ;
X = [7/2, 6/7, 8/4, 5/3, 4/6, 3/8, 2/5, 1/1] ...
```

Removendo Permutações

Se meu tabuleiro é 8x8, e eu tenho de encaixar 8 rainhas, tenho de **encaixar uma rainha por linha**.

```
eightqueens(X) :- X = [1/_/_/_/_/_/_/_], legal(X).
```

```
?- eightqueens(X).
X = [1/4, 2/2, 3/7, 4/3, 5/6, 6/8, 7/5, 8/1] ;
X = [1/5, 2/2, 3/4, 4/7, 5/3, 6/8, 7/6, 8/1] ;
X = [1/3, 2/5, 3/2, 4/8, 5/6, 6/4, 7/7, 8/1] .
```

Eliminando Redundâncias

Se agora `X` ficará, necessariamente, entre 1 e 8, então **suas verificações ficaram redundantes**.

```
...
nocheck(X/Y, [X1/Y1 | T]) :-
    Y \= Y1,
    abs(X-X1) \= abs(Y-Y1),
    nocheck(X/Y, T).

...
legal([X/Y | T]) :- legal(T),
    member(Y, [1,2,3,4,5,6,7,8]),
    nocheck(X/Y, T).
```

Antes:

```
?- time(eightqueens(X)).
% 16,334,000 inferences, 0.860 CPU in 0.860 seconds (100% CPU, 18983468 Lips)
```

Depois:

```
?- time(eightqueens(X)).
% 9,596 inferences, 0.001 CPU in 0.001 seconds (100% CPU, 8778703 Lips)
```

Agora sim! 🎯